PROJECT DEEP-DIVE & INTERVIEW MASTER DOCUMENT

IW Computer Vision Services

An AWS-native intelligent document-understanding platform for benefits documents — the problem, the architecture, the ML/GenAI design, and how to present it

AWS Serverless | Textract + Comprehend | Grounded GenAI | Custom PyTorch Transformer | Factory Design Pattern | MLOps / Governance

---

Prepared by Sheriff (Mohammad Sheriff Mehmood) | Senior Data Scientist & AI/ML Engineer, ValueLabs (embedded with Businessolver Data Science)
Compiled from repository documentation, architecture artifacts, notebooks, console screenshots and project briefs | Edition: 28 Aug 2026

All performance numbers in this document are cohort-specific documented project results, not universal production guarantees.

1 What the Project Is & What Problem It Solves

The business problem, why a naive approach fails, and the product answer

The problem

Benefits administration runs on documents — carrier PDFs, scans and images — that are **not clean database records**. The same business concept (a plan attribute, a claim line item, a dental procedure code) shows up in different positions, different table layouts, different fonts and different carrier-specific formats. Two failure modes show up if you attack this the obvious ways:

- **A purely hard-coded parser** is cheap on day one but expensive forever — it breaks every time a carrier changes a layout, and maintaining N carrier-specific parsers does not scale.
- **A purely generative (LLM-only) solution** is fast to build but risky in production — benefits decisions need **traceable evidence**, and a model that "sounds right" is not the same as a model that is verifiably right.

The product answer — IW Computer Vision Services (CVS)

CVS is an intelligent document-understanding (IDU) platform that turns carrier PDFs/scans/images into structured, reviewable data (plan attributes, claim details, dental line items) using a **hybrid extraction decision**:

Deterministic rules — stable layouts | OCR + Textract queries — structured evidence | Custom ML — table / document selection | GenAI — only inside an evidence-grounded validation boundary

A React console lets operations teams review, correct, author, train, evaluate and monitor outcomes — so this is a full operational product, not a notebook prototype.

SIMPLE SUMMARY (FOR A NON-TECHNICAL LISTENER)

It's a high-assurance document factory: a user uploads a PDF or image, the platform figures out its layout and type, picks the best extraction method for that document, proves exactly where each extracted value came from, and hands structured data to a human reviewer and downstream systems.

HINGLISH VERSION

Document upload hota hai → system usko OCR se padhta hai → document type identify karta hai → best model/strategy choose karta hai → result ko source words ke saath validate karta hai → phir reviewer aur downstream system ko data milta hai.

Business value delivered

VALUE	HOW IT'S ACHIEVED
Less manual document reading	Automated OCR + classification + extraction; humans only review uncertainty and business-critical exceptions.
Carrier-aware change management	New/changed layouts are handled via versioned templates & strategies — not by rewriting a monolith.
Full auditability	Every extracted value ties back to Textract Word IDs, bounding boxes, model version, strategy version and evaluation evidence.

Core vocabulary (use these words precisely in an interview)

TERM	MEANING IN CVS
OCR	Converts page pixels into words, lines, forms, tables, queries and geometry. AWS Textract is the primary OCR/layout engine.
Classification	Predicts document type & routing context. AWS Comprehend custom classification is the primary capability.
Extraction strategy	A versioned instruction set describing how a document / field group should be extracted.
Target	A named output contract (e.g. a plan field or a line-item group) with ownership & validation rules.
Grounding	Every GenAI answer references pre-existing OCR Word IDs; server-side logic verifies them before a value is trusted.

2 Top-Down Architecture

Six layers that separate public interaction, secure intake, async work, intelligence, and delivery

The architecture deliberately separates concerns so a large PDF or a slow OCR call never holds a browser request open. **Design principle:** the queue carries a small *job reference*, not the document bytes — documents live in S3, status lives in DynamoDB, and workers scale independently. This is a resilient, well-known serverless pattern for document workloads.

1 Users & Systems

Partner system / React console — used by admins, reviewers, analysts to start upload, authoring, testing and review work.

↓

2 Secure API & Intake

API Gateway (SigV4 authorizer + WAF) | Upload Lambda → presigned URL | Queue Lambda → job + task request

↓

3 Asynchronous Processing

S3 staging (source documents) | DynamoDB (job state) | SQS queues (retry + DLQ) | SQS worker — containerized Lambda

↓

4 Document Intelligence

Normalize PDF / image pages | Textract — OCR / layout | Comprehend — classification | Template config (DynamoDB)

Strategy Factory — resolve type + version

↓

5 Extraction & Persistence

Strategy execution: Textract / ML / GenAI | S3 artifacts + DynamoDB documents

↓

6 Review, Events & Learning

React review console — correction + authoring | Outbound event: SQS / EventBridge | Downstream benefits & claims consumers

Why each layer exists (failure isolation)

LAYER	WHY IT'S NEEDED	FAILURE ISOLATION IT BUYS
Users & systems	Starts upload, authoring, testing, review work.	Browser stays responsive; no long-lived AWS credentials client-side.
API & intake	Controlled entry points via API Gateway, WAF, SigV4/session controls.	Auth & request validation happen before any processing.
Async processing	S3 holds bytes, DynamoDB owns state, SQS decouples bursts, DLQ retains unhandled work.	Retries don't duplicate storage or lose status.
Intelligence	OCR, classification, templates and the strategy factory choose the correct extraction logic.	One bad document never blocks unrelated work.
Delivery & learning	Results persisted, surfaced in UI, emitted to consumers, then reused for supervised improvement.	Review corrections become governed learning evidence.

3 Document Runtime Flow — Upload to Trusted Output

The single most important flow to narrate in a system-design interview

1. **Upload** — Client requests a short-lived presigned S3 URL and uploads bytes directly to S3 (no backend proxy of file bytes).
2. **Queue** — API stores a `PENDING` job and sends a small job reference to SQS.
3. **Prepare** — Worker downloads the file, validates MIME type/page count, rasterizes or compresses when needed.
4. **Understand** — Textract returns words, tables, forms, queries and geometry; the raw JSON response is retained for replay/debug.
5. **Classify** — Comprehend predicts document type, confidence and a version signal.
6. **Route** — The Strategy Factory resolves the correct template and strategy revision for this document.
7. **Extract** — Runs deterministic rules, a Textract adapter, the target-table selector, custom ML, or grounded GenAI — whichever the strategy specifies.
8. **Validate** — Checks schema, target ownership, known Word IDs, confidence and spatial evidence before anything is trusted.
9. **Persist** — Writes the normalized document record, artifacts, task status and observability data.
10. **Deliver** — Publishes a completion/failure event; the React review console and downstream consumers receive traceable output.

What happens when something fails

FAILURE TYPE	BEHAVIOR
Retryable / transient error	Worker re-queues the work; job record captures state and diagnostics.
Terminal / unhandled error	Message is isolated in a Dead-Letter Queue (DLQ) for investigation — never silently dropped.
OCR / classification issue	Raw Textract responses and intermediate artifacts stay in S3 so an engineer or reviewer can reproduce the exact issue.
Extraction / GenAI validation failure	System rejects unsupported schema, invalid targets or unknown Word IDs rather than trusting a plausible-looking value.

INTERVIEW INSIGHT

Don't describe this as "an OCR model." It's an end-to-end document-intelligence *system* that treats OCR as evidence, manages asynchronous work, uses a routing layer, enforces output contracts, and closes the loop through reviewer feedback.

4 Low-Level Design — Component Catalog & What Each Technology Means

Every moving part, its responsibility, why it matters, and the plain-English meaning of the technology

Platform foundation components

COMPONENT	PRIMARY RESPONSIBILITY	WHY IT MATTERS
API Gateway	Public REST boundary for upload, queueing, console operations, templates, strategies, metrics.	Centralizes authorization, WAF protection, CORS/routing and API versioning.
Upload Lambda	Creates short-lived presigned S3 upload URLs.	Browser uploads large bytes directly to S3; backend avoids proxying files.
Queue Lambda	Creates/updates the processing job and places a job reference on SQS.	Fast request response with a durable async handoff.
S3 staging bucket	Holds incoming PDFs/images; a durable bucket also retains normalized pages, Textract JSON and artifacts.	Separates document payload from orchestration state; supports replay/debug.
DynamoDB	Stores jobs, classified documents, templates, strategies, models, test results and tasks.	Low-latency state and version metadata for operational workflows.
SQS + DLQ	Buffers inbound processing and isolates unhandled failure messages.	Absorbs bursts, enables retry policy, protects workers, preserves failed work.
SQS worker Lambda	Downloads the document, calls the processing pipeline, updates state, emits completion events.	Elastic per-document processing without a request timeout.
Textract	OCR/layout evidence: pages, words, tables, forms, queries, geometry, adapters.	Provides the source material every deterministic/ML/GenAI method builds on.
Comprehend custom classifier	Predicts document type and routing confidence.	Routes a heterogeneous document set to the right carrier/document strategy.

Intelligence, UI & delivery components

COMPONENT	PRIMARY RESPONSIBILITY	WHY IT MATTERS
Document template	Defines document-type configuration, fields, target contracts and extraction settings.	Moves behavior into governed configuration instead of scattered conditional code.
TextractStrategyFactory	Resolves an extraction strategy based on classified type, template, carrier/model revision and runtime context.	Factory pattern isolates algorithm choice; makes future strategies composable.
Extraction strategy	Executes queries, forms, table extraction, adapter calls, custom algorithms, composite logic or GenAI.	Each document family evolves independently without destabilizing others.
Validation & rehydration	Checks schema, target ownership, Word IDs, evidence, confidence and geometry; reconstructs final fields.	Prevents a weak or ungrounded response from becoming system-of-record data.
Outbound queue/event	Notifies downstream benefits/claims consumers of completion or failure.	Decouples consumers from processing latency and deployment cadence.
React admin console	Review source blocks, correct values, author templates/strategies, choose datasets, run tests, view metrics.	Human-in-the-loop quality — an operational product surface, not a notebook prototype.
Step Functions	Orchestrates longer model-training/evaluation/testing workflows and state transitions.	Visible, retryable workflow coordination beyond a single Lambda invocation.
ECS/Fargate	Runs heavier containerized ML/GenAI training or evaluation tasks.	Avoids Lambda runtime limits for CPU/memory/runtime-intensive jobs.
CDK	Infrastructure-as-code for API, Lambda, S3, SQS, DynamoDB, ECS, Step Functions, IAM, Secrets, observability.	Makes environment infrastructure reviewable and repeatable.

Repository map

AREA	CONTENTS
`lambda/src`	Python REST resources, services, domain objects, repositories, AWS abstractions, extraction strategies, model integrations.
`cdk/app`	Python CDK constructs for code/infrastructure stacks and per-environment configuration.
`webapp/src`	React + TypeScript console for operations, review, authoring, training/test launches and metrics.
`notebooks`	Exploratory R&D — data analysis, clustering, target-table ML, benchmarking, experiment design. Not all notebooks are production runtime.
`documentation/`	Architecture flows, API/design contracts, deployment guidance, experiment documentation.

5 Hybrid Intelligence & the Factory Design Pattern

Why the system doesn't force every document through one model

The factory pattern separates **what** needs to be extracted from **how** to extract it — a practical way to scale a document product across changing carriers and document families, instead of one giant if/else block in the worker.

Decision ladder (method → when to use it)

Textract forms / queries / tables

Best for: known fields, visually structured pages | Strength: fast OCR evidence + geometry | Control: query design, schema, response validation

Deterministic strategy

Best for: stable carrier layout / unambiguous header | Strength: predictable, explainable | Control: versioned strategy + unit/integration coverage

Textract adapter

Best for: document family where an adapter is available | Strength: managed domain adaptation | Control: adapter/version tied to strategy

Custom table selector

Best for: multiple tables, need exactly one business-relevant table | Strength: ranks tables from semantics/layout/structure | Control: document-level split, fallback heuristic, calibration

Grounded GenAI

Best for: semantic ambiguity, variable language, hard carrier-specific mapping | Strength: flexible interpretation with evidence references | Control: strict JSON/Word-ID/target validator + audit artifacts

Composite strategy

Best for: several methods must contribute non-overlapping targets | Strength: reuse + incremental migration | Control: target collision + dependency-graph checks

How the factory actually works

- **Input:** classified document type, template configuration, carrier context, strategy/model version, runtime job state.
- **Resolver:** `TextractStrategyFactory` selects the strategy implementation instead of a giant conditional block.
- **Execution:** a strategy returns fields and/or table groups under defined output targets; a composite strategy can combine safe, non-overlapping outputs.
- **Guardrails:** a model revision belongs to exactly one strategy; target ownership/collisions are checked; lifecycle versions are controlled; only eligible revisions enter runtime selection.

WHY A HIRING PANEL CARES

This is design-for-change. It demonstrates abstraction boundaries, version governance, testability, extensibility, and an explicit trade-off between deterministic accuracy and model flexibility — exactly what senior ML/platform interviews probe for.

6 ML Workstream — Target-Table Selection

Choosing the one business-relevant table out of many candidate tables on a page

Benefits documents often contain several tables — rates, notes, plan summaries, exceptions, line-item benefits — but the downstream system may need only the line-item table. Picking the wrong table makes every cell-level extraction look wrong even when OCR itself is perfect.

STAGE	METHOD
Candidate creation	Profile every Textract table candidate: headers, aliases, nearby context, row/column count, amount/date/description signals, repeating-row patterns.
Baseline	Weighted heuristic scoring header match, structural patterns and business keywords — kept as the explicit fallback while models are validated.
Training labels	Documented dental dataset: 200 documents, 653 tables, 4 categories → 200 positive tables, 453 negative.
Leakage-safe evaluation	Split by **document ID**, not table rows — group-aware train/val/test splits so tables from one document never leak across cohorts.
Models explored	Logistic Regression, Random Forest, XGBoost, SVM, HistGradientBoosting — optimized for F1, compared on precision/recall, ROC-AUC, average precision, document-level top-1 accuracy.
Runtime plan	Build the same features online, score candidates, apply threshold/calibration, pick the top table when confident, else fall back to heuristic/manual review — and log the selection evidence.

DOCUMENTED RESULT — USE WITH CONTEXT

F1 / precision / recall of **1.0** for dental table selection on the documented evaluation set (200 documents, 653 tables, 4 categories). Present this as a **cohort-specific offline result**, not an unrestricted production guarantee.

Unsupervised clustering (R&D)

The notebooks also describe an exploratory discovery path: build document/block features from layout, semantic embeddings and entity frequency → reduce dimensionality (e.g. UMAP) → cluster with KMeans/DBSCAN → assess silhouette and Davies-Bouldin scores → inspect clusters for new document families or label-efficiency opportunities.

CLAIM BOUNDARY

Say "built/evaluated an R&D pipeline" for clustering and exploratory table work — unless a deployed runtime, reproducible experiment and release evidence confirm production use.

7 Grounded GenAI — Flexibility Without Losing Evidence

How the system uses an LLM without letting it hallucinate a benefits value

GenAI is most valuable where fields are semantically complex or layouts vary enough that a fixed rule becomes brittle. CVS limits that flexibility with a hard evidence contract: **the model must identify source Word IDs already emitted by Textract** — it cannot introduce new text.

1. **Textract evidence** — Word IDs, cells and geometry are already extracted before the LLM is called.
2. **GenAI prompt** — contains only relevant document/table/cell evidence, the output schema, field definitions, examples and existing Word IDs.
3. **Strict validator** — server code checks response shape, list types, target ownership, and every returned Word ID against the original Textract response. Unknown IDs / malformed fields are rejected and logged.
4. **Rehydrated output** — trusted value + source blocks, bounding boxes and confidence are reconstructed from the *original* OCR words for UI review.

The model may select evidence — server-side validation decides what becomes a trusted field.

Why this matters for regulated / high-impact data

RISK	GROUNDED DESIGN RESPONSE
Hallucinated value	A model cannot introduce a value without pointing to an existing OCR Word ID; server validates it.
Wrong field mapping	Target/schema ownership rules and field-level validation constrain placement.
No audit trail	Stored artifacts preserve raw OCR, chosen Word IDs, geometry, model/strategy revision and test results.
Layout change	Carrier profile/strategy revision can be versioned and evaluated without rewriting unrelated document types.
Model regression	Separate test cohorts, evaluation artifacts and a promotion lifecycle prevent casual production replacement.

HOW TO SAY IT IN AN INTERVIEW

"I did not treat an LLM answer as truth. I used GenAI as a constrained evidence selector, then applied deterministic server-side validation and rehydration so every business value remained traceable to the original document."

8 Custom Model — PyTorch Multi-Query Pointer Transformer

A learned alignment layer that sits on top of Textract to fix table/field misalignment

Textract is excellent at OCR and geometry, but a generic `TABLE` response struggles on real EOB layouts: split/double headers, merged cells, wrapped text, remarks/total rows, multiple claim tables, and row/line association errors. The custom model addresses exactly this — it does **not** replace Textract end-to-end; it replaces or augments the table-to-field **alignment** layer, while Textract remains the OCR/geometry source of truth.

How it works

1. **Text embeddings** — SentenceTransformer `all-MiniLM-L6-v2` creates 384-dimensional embeddings for table words and natural-language field queries.
2. **Geometry features** — each token gets 11 spatial features derived from the Textract bounding box/position.
3. **Concatenate & project** — text + geometry are combined and projected to 256 dimensions.
4. **Self-attention encoder + FFN** — heads/layers tuned via validation performance.
5. **Query-to-token scoring** — every OCR token is scored against each field-order query (e.g. `amount_1`, `description_3`), plus a separate NULL score representing field absence.
6. **Per-query thresholds** — token probability threshold (tau) and a NULL-margin (delta), calibrated per query from validation precision-recall curves.
7. **Selected span + union bbox** — final value, merged bounding box and confidence, ready for the UI.

Document template fields are expanded by claim order — roughly **100 field-order queries** for the Aetna example, and a separate saved artifact records **60 queries** for an Anthem run. Training labels come from human annotations in DynamoDB, aligned to OCR words by bounding-box overlap (IoU ≥ 0.05 marks a word positive during alignment).

Comparison of approaches used together in CVS

APPROACH	STRENGTH	MAIN WEAKNESS	ROLE IN CVS
Textract TABLES	Fast OCR, cells, tables, Word IDs, geometry.	Generic topology can be brittle on irregular/multi-table EOB layouts.	Evidence provider & baseline table parser.
Carrier rules / fuzzy headers	Explainable; strong for stable known formats (double headers, totals).	Grows with layout variation; hard to maintain across carriers.	Deterministic strategy / fallback.
PyTorch pointer Transformer	Learns field-to-token alignment from text + layout; explicit NULL handling and evidence boxes.	Needs labeled data, per-carrier validation, monitoring, model lifecycle.	Custom extraction/augmentation for difficult tables.
Grounded GenAI	Flexible semantics, carrier-specific mapping, selects source Word IDs.	Cost/latency and prompt/model governance required.	Configured option for complex extraction targets.

Training & calibration pipeline

STAGE	WHAT HAPPENS
Build samples	Align labeled field boxes to Textract words; store text/geometry/query vectors and token masks.
Train	PyTorch AdamW, gradient clipping, dropout, attention layers; track loss/token/presence metrics per epoch.
Select best model	Choose by **validation** token F1 (never the test set); save state dict, hyperparameters, query list, split IDs, thresholds.
Calibrate tau / delta	Per-query token probability threshold and NULL-margin, from validation precision-recall behavior.
Inference	Score all words per query, apply tau/delta, select token spans, assemble response from original text/boxes.

Five automated postprocessing QA checks

1. **BBox present** — does every high-confidence non-NULL field have a bounding box?
2. **Column consistency** — do repeated fields of the same type stay in the same logical column?
3. **Row order** — are `amount_1, amount_2...` ordered top-to-bottom, not swapped?
4. **IoU vs. gold** — does predicted geometry overlap the human-labeled box by at least the required IoU?
5. **NULL alignment** — when gold says no value exists, does the model also abstain (and vice versa)?

DOCUMENTED SPATIAL QA RESULT — AETNA POSTPROCESSING BATCH, 135 DOCUMENTS

BBox present: 100.0% | Column consistent: 100.0% | Row order: 100.0% | IoU ≥ 0.5 pass: 90.4% | NULL aligned: 91.1% | Mean IoU: 0.9607

State this as an **offline notebook cohort result** — 13/135 documents had a spatial mismatch and 12/135 had a NULL-alignment mismatch under this rule; the mean hides those failures.

ACCURATE WORDING FOR INTERVIEWS

Say: "*I developed a custom PyTorch Transformer to replace/augment the Textract table-alignment and field-extraction layer, while retaining Textract OCR geometry as evidence.*" Do not say "I replaced AWS Textract completely."

9 Metrics, Statistics & Testing Rigor

How to read the numbers correctly, and why this is NOT A/B-tested (yet)

The 5-level metric stack

LEVEL	METRICS	WHY IT EXISTS
1. Routing	Class accuracy, macro/micro F1, precision, recall, confusion matrix	Wrong document type routes to the wrong strategy before extraction even begins.
2. Field extraction	Per-field TP/FP/FN, precision, recall, F1, exact/normalized match	Measures whether business values are correct.
3. Presence / NULL	Presence precision/recall/F1, false extraction rate, missed-field rate	Avoids claiming a field exists when it's absent, or missing a real value.
4. Spatial evidence	Bounding-box IoU, bbox-present rate, row order, column consistency	Verifies the result is visually grounded in the right table location.
5. Operations	Perfect-document rate, review rate, latency, cost/document, retries/failures	Measures whether the product is useful and safe in production.

Formulas worth knowing cold

METRIC	MEANING
Precision	TP / (TP + FP) — of what the system returned, how much was right?
Recall	TP / (TP + FN) — of the true values, how much did the system recover?
F1	2 × P × R / (P + R) — one balance score; never a substitute for showing P and R separately.
IoU	Area(predicted ∩ gold) / Area(predicted ∪ gold). The project uses IoU ≥ 0.5 as a spatial pass rule.
Macro vs. micro	Macro averages document-level values equally; micro pools all counts. Report both when sizes vary.

CRITICAL NUANCE — DON'T MIX COHORTS

Three different metric surfaces exist (classification dashboard, extraction model dashboard, threshold dashboard) and they answer **different questions**. Example: an extraction dashboard reported field F1 = 0.931, but a document-quality threshold view of the same model reported F1 = 0.545 — because it treats "perfect document" as the positive class and was read at threshold 0. **Never say "the model has 54.5% F1"** from that graph — it answers a different question than the 0.931 field-extraction number.

Did the project do A/B testing?

Not in the evidence reviewed. The project has stratified train/val/test splits, hyperparameter tuning, threshold calibration, holdout retests, field/document metrics and strategy/model versioning — all of that is rigorous **offline evaluation and MLOps**, not a live experiment. A true online A/B test needs randomized document assignment to control/treatment, persistent assignment logs, pre-defined success/guardrail metrics and a statistical comparison of live outcomes — none of that was found. A "Compare Model" button or a versioned strategy picker in the UI is **versioned routing**, not an A/B experiment.

The path to a valid live comparison (say this to sound senior)

PHASE	ACTION
1. Frozen offline benchmark	Run baseline, PyTorch, and GenAI versions on the exact same locked document IDs/labels.
2. Paired error analysis	Compare each document/field side by side, grouped by carrier, page count, table shape.
3. Statistical confidence	Bootstrap CIs at document level; paired tests (e.g. McNemar) for correctness events.
4. Shadow mode	Run candidate in parallel; baseline still drives decisions; log both + reviewer result.
5. Live A/B (only if approved)	Stable hash of document ID assigns control/treatment; pre-registered metric, guardrails, stopping rule.
6. Promote / rollback	Promote only if primary metrics improve without breaking safety/cost/latency guardrails.

Documented figures — always attach the qualifier

REPORTED FIGURE	REQUIRED QUALIFIER
Tens of thousands of documents/week across 20+ classes	Cite environment period and class definition before using externally.
0.8–0.9 classification F1	Attach class distribution, evaluation split, threshold and date.

0.7–0.9 field extraction performance	Don't collapse heterogeneous fields into one universal number.
<10% manual review, <5 sec/document	Define workload, percentile, and clock start/end.
~$0.03/document GenAI cost	Depends on prompt, tokens, model, OCR and document mix; state the period.
90%+ grounded GenAI vs. 65–70% rule baseline	Carrier/cohort-specific; state fields, sample size, evaluation protocol.
Target-table F1/P/R = 1.0	Offline dental cohort only (200 docs / 653 tables); report the held-out split and leakage controls.

RESUME SAFETY RULE

Never put an accuracy/cost/time number on a resume unless you can explain the denominator, cohort, split, time period and your direct contribution. Strong candidates make metrics *more* credible by stating their measurement method.

10 Current Delivery — Async GenAI Model-Test Feature

The most recently implemented capability, end to end

This feature connects an extraction-strategy "Test" button in the React console to a long-running GenAI evaluation workflow. It's intentionally asynchronous: a model test processes a whole test cohort and produces artifacts, which cannot fit inside one HTTP request/response cycle.

LAYER	OBSERVED IMPLEMENTATION ROLE
React client/workspace	Calls the model-test endpoint from the extraction-strategy workspace; registers async task status and refreshes result state.
Lambda API resource	Exposes `POST /v1/extraction-strategies/{strategy}/{version}/models/{model}/{version}/test` and delegates to the service layer.
Service validation	Checks async repository, context, GENAI model kind/ownership, non-empty test dataset, and no conflicting in-progress evaluation before starting.
Step Functions	Starts a named GenAI model-test state machine using an environment-configured ARN and execution input.
ECS/Fargate worker	Runs isolated model evaluation on test documents, persists artifacts/results, completes lifecycle state.
DynamoDB / S3 / UI	Task/evaluation/result records and artifacts support status, troubleshooting, and the user-facing test outcome.

WHY A 403 HAPPENED DURING ROLLOUT — AND WHY THAT'S NOT A CODE BUG

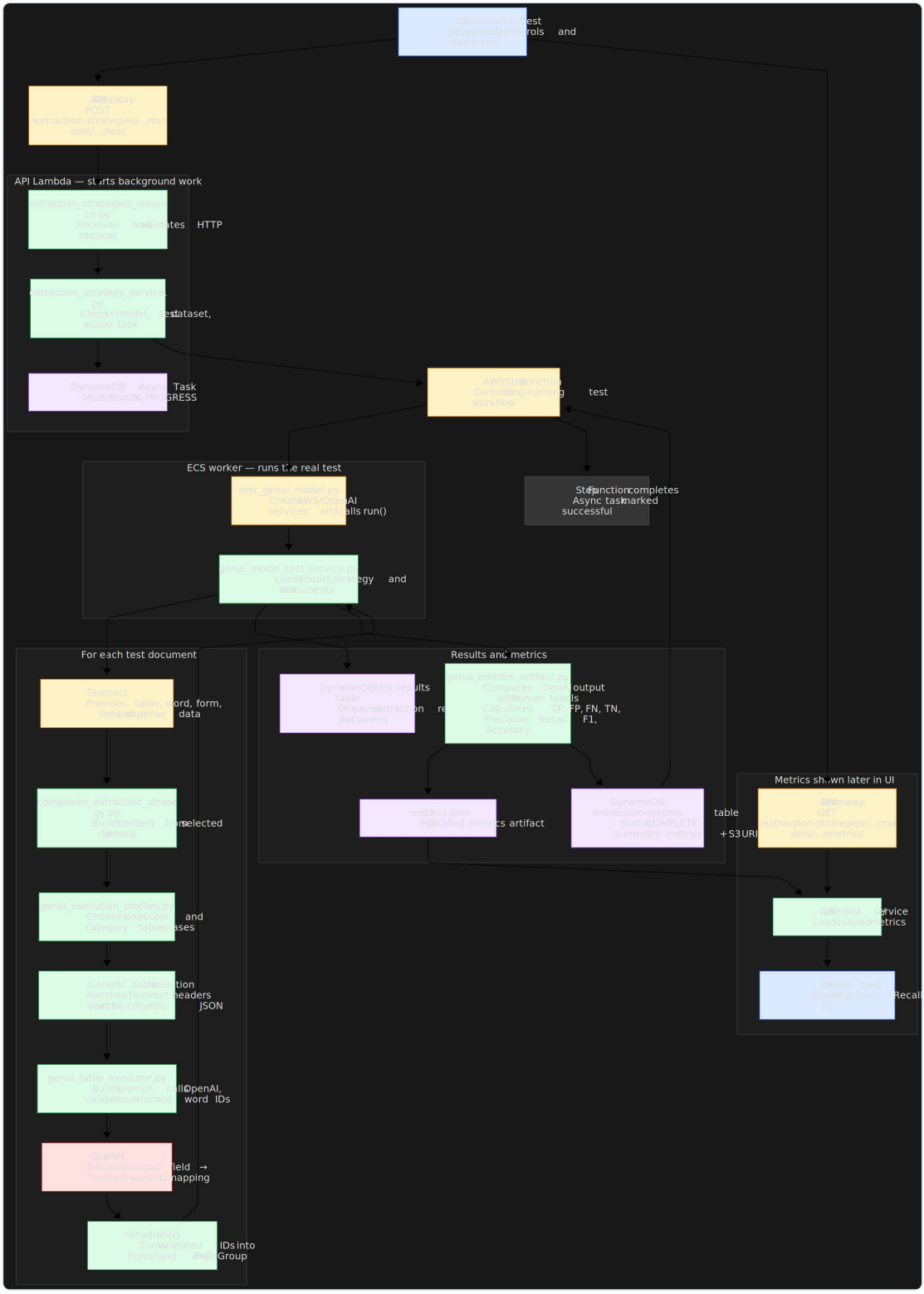
Before the new API route/stack is deployed to DEV, API Gateway cannot authorize or route the new resource. A 403 at that stage is a **deployment/environment contract issue**, not evidence that the browser button or worker logic is wrong.

Deployment & test gate status

- CDK synth/list demonstrated the dev environment resolves the expected infrastructure/code stacks.
- CDK diff reviewed in CI: state machine, Fargate task definition, IAM, security groups, API route, Lambda env vars, webapp artifact.
- **Current status:** implementation and local diff/test preparation exist; DEV deployment and a live end-to-end execution must be completed before calling it "deployed."
- Integration test is opt-in and environment-safe: real Step Functions ARN, SSO/profile, test documents, permissions, cleanup approach required.

10a Reference Diagram — Detailed Async Model-Test Flow

React button → API Gateway → Lambda service → Step Functions → ECS/Fargate worker → DynamoDB/S3 → UI result card (source: supplied architecture diagram, reproduced at full detail for reference)



11 Human-in-the-Loop, Lifecycle & Governance

Why this is a governed ML platform, not just a model

Strategy / model lifecycle

1. **DRAFT** — configuration can evolve while the strategy/model is being prepared.
2. **DATASET / TRAIN / TEST** — explicitly selected cohorts and async tasks create reproducible artifacts.
3. **EVALUATE** — compare metrics and inspect failure examples; never promote on one aggregate number.
4. **SEALED** — immutable revision after required work is complete; audit + reproducibility boundary.
5. **PROMOTE** — makes an approved, evaluated revision eligible for runtime selection.

Invariant: only sealed, evaluated revisions can be promoted into document runtime selection.

Governance model

AREA	CONTROL	PRACTICAL OUTCOME
Data	Human corrections/labels, distinct train/val/test cohorts, dataset identifiers.	Honest evaluation and a trail from output back to data.
Model	Model IDs, strategy/model versions, kind, owner, test status, metric artifacts, immutable sealed revisions.	A runtime result can identify exactly what logic produced it.
Extraction	Output targets, collision detection, dependency-graph validation, field/schema contracts.	Composite logic cannot silently overwrite another component's result.
GenAI	Prompt/profile/schema/evidence contract, known Word-ID check, rejected-output handling.	Flexible language processing never bypasses deterministic trust controls.
Operations	Job/task state, SQS retry, DLQ, logs, artifacts, error status, async execution records.	Failures are observable and recoverable rather than becoming missing data.

Security architecture

- API Gateway as the controlled edge — SigV4 authorization + WAF protection.
- Presigned S3 URLs for browser upload — reduces backend file-proxy load, limited in scope/time.
- Admin console uses same-origin/session-oriented access — no long-lived AWS credentials in browser code.
- IAM roles scoped per service; Secrets Manager for protected configuration; CDK makes permissions reviewable as code.
- AWS SSO + named profile for developer environment access — separates local identity from application runtime roles.

SYSTEM-DESIGN FRAMING TO REPEAT IN INTERVIEWS

"The platform favors least privilege, short-lived access, durable state, explicit retries, immutable audit artifacts and role-separated orchestration — the same principles used in production ML systems at scale."

12 FAANG-Level Resume Bullets

Data Scientist / AI-ML Engineer framing — use only what matches your actual contribution

DOCUMENT AI PLATFORM

Designed and implemented an AWS-native intelligent document-understanding pipeline using S3, SQS, Lambda, DynamoDB, Textract, Comprehend, Step Functions and ECS/Fargate to convert unstructured benefits documents into traceable structured data.

EXTENSIBLE DESIGN

Built a factory-based extraction architecture that resolves versioned carrier/document strategies at runtime, replacing monolithic hard-coded routing with composable deterministic, adapter, custom ML and GenAI components.

GROUNDED GENAI

Implemented evidence-grounded GenAI extraction in which model outputs reference Textract Word IDs; added schema, ownership and source validation before rehydrating reviewer-visible values with geometry and confidence.

ASYNC EVALUATION

Delivered an asynchronous model-test path from React through API/Lambda validation to Step Functions and Fargate, persisting evaluation tasks, artifacts and results for reproducible model governance.

TABLE ML

Developed and evaluated document-level target-table selection features using header semantics, layout/context and structural signals; used grouped splits to prevent document leakage and benchmarked classifiers including XGBoost/HistGradientBoosting against a heuristic fallback.

MLOPS / QUALITY

Defined a governed strategy/model lifecycle with dataset cohort separation, versioned artifacts, field-level evaluation, promotion gates and operational failure handling (retries, DLQ, audit records).

CUSTOM TABLE EXTRACTION

Developed a PyTorch multi-query pointer Transformer combining 384D semantic embeddings with 11D layout features to align EOB table fields to OCR tokens, reducing dependence on brittle row/line heuristics.

VALIDATION FRAMEWORK

Implemented a multi-layer evaluation framework spanning token/field P-R-F1, per-query NULL calibration, document quality, row/column invariants and gold-bounding-box IoU.

Impact bullets — attach only verified metrics

IF YOU CAN VERIFY	USE THIS FORMAT
Target-table evaluation	Achieved [F1/precision/recall] on a held-out, document-grouped dental table-selection cohort of [N documents / N tables], improving reliable line-item extraction by selecting the correct source table.
Operational automation	Reduced manual-review rate to [X%] and processing latency to [Y sec/doc] across [volume/time window], while retaining evidence-backed reviewer correction for exceptions.
GenAI quality	Improved carrier-specific extraction from [baseline] to [result] on [fields, cohort, date], using Word-ID grounding and schema validation to preserve auditability.
Cost	Delivered a GenAI-assisted flow at approximately [$X/document] for [model, page/token/document mix], with [latency] and [quality] guardrails.

DO NOT OVERCLAIM

Do not write "replaced AWS Textract with a CNN" unless a production deployment, benchmark dataset, reproducible training run and side-by-side metric exist. Safer and equally strong: "*researched/evaluated custom CV or CNN alternatives for table detection, with deployment pending validation.*"

13 Interview Narrative — Say It Like a Senior Engineer

Two ready-to-use pitches, plus Q&A for the toughest follow-ups

30-second pitch

"I worked on an AWS-native document-intelligence platform for benefits documents. It ingests PDFs asynchronously, uses Textract for OCR/layout and Comprehend for document routing, then a factory selects a versioned extraction strategy — deterministic, ML, adapter or evidence-grounded GenAI. The important part is trust: results are validated against source Word IDs and shown to reviewers with geometry, so the system scales automation without losing auditability."

90-second system-design pitch

"The challenge was carrier variation: templates and table layouts change, so a single hard-coded parser was not maintainable. I separated payload, state and work: S3 stores documents/artifacts, DynamoDB owns job and model metadata, SQS decouples processing, a worker runs OCR/classification, and a strategy factory routes to the right algorithm. For difficult semantics, the GenAI path is grounded in Textract Word IDs, and the backend validates IDs, schema and target ownership before output is persisted. The console supports review and correction, while versioned datasets, strategy/model revisions, Step Functions and Fargate give us a repeatable test/evaluation lifecycle. I'd measure quality at classification, table-selection, field and evidence levels — not one generic accuracy number."

Likely technical questions and the right answer direction

QUESTION	ANSWER DIRECTION
Why not use an LLM for every document?	Cost, latency, nondeterminism and auditability. Use deterministic/OCR methods where reliable; reserve grounded GenAI for semantic ambiguity, and validate every output.
How do you prevent data leakage?	Split table-selection/evaluation data by document ID/group; keep separate train/val/test cohorts; version dataset membership.
How would you handle a new carrier layout?	Classify/cluster the new samples, author a template/strategy revision, collect labels, test on a held-out cohort, compare against baseline, seal only after gates pass, then promote.
How do you debug an incorrect field?	Trace job/task ID → model/strategy/template revision → raw Textract artifact → selected evidence/Word IDs → validator output → UI source block, then classify root cause (OCR, routing, selection, prompt, schema, target contract).
What makes the system scalable?	Direct-to-S3 upload, SQS buffering, stateless workers, persisted state, retries/DLQ, and asynchronous Step Functions/Fargate for long-running work.

What to emphasize for FAANG interviews

- Start with the **business constraint and trade-offs**, not a cloud-service list.
- Explain evidence-grounding as a **product-quality mechanism**, not merely prompt engineering.
- Use leakage-safe evaluation and per-field/operational metrics to demonstrate **ML rigor**.
- Show how the factory/version model lowers the **marginal cost** of supporting the next carrier or document type.

FINAL TAKEAWAY

IW Computer Vision Services is best presented as a **trustworthy document-intelligence platform**: scalable ingestion plus hybrid ML/GenAI plus a governed human feedback loop. The strongest differentiator is the combination of system design, ML rigor, and evidence-backed AI output — not any single model.